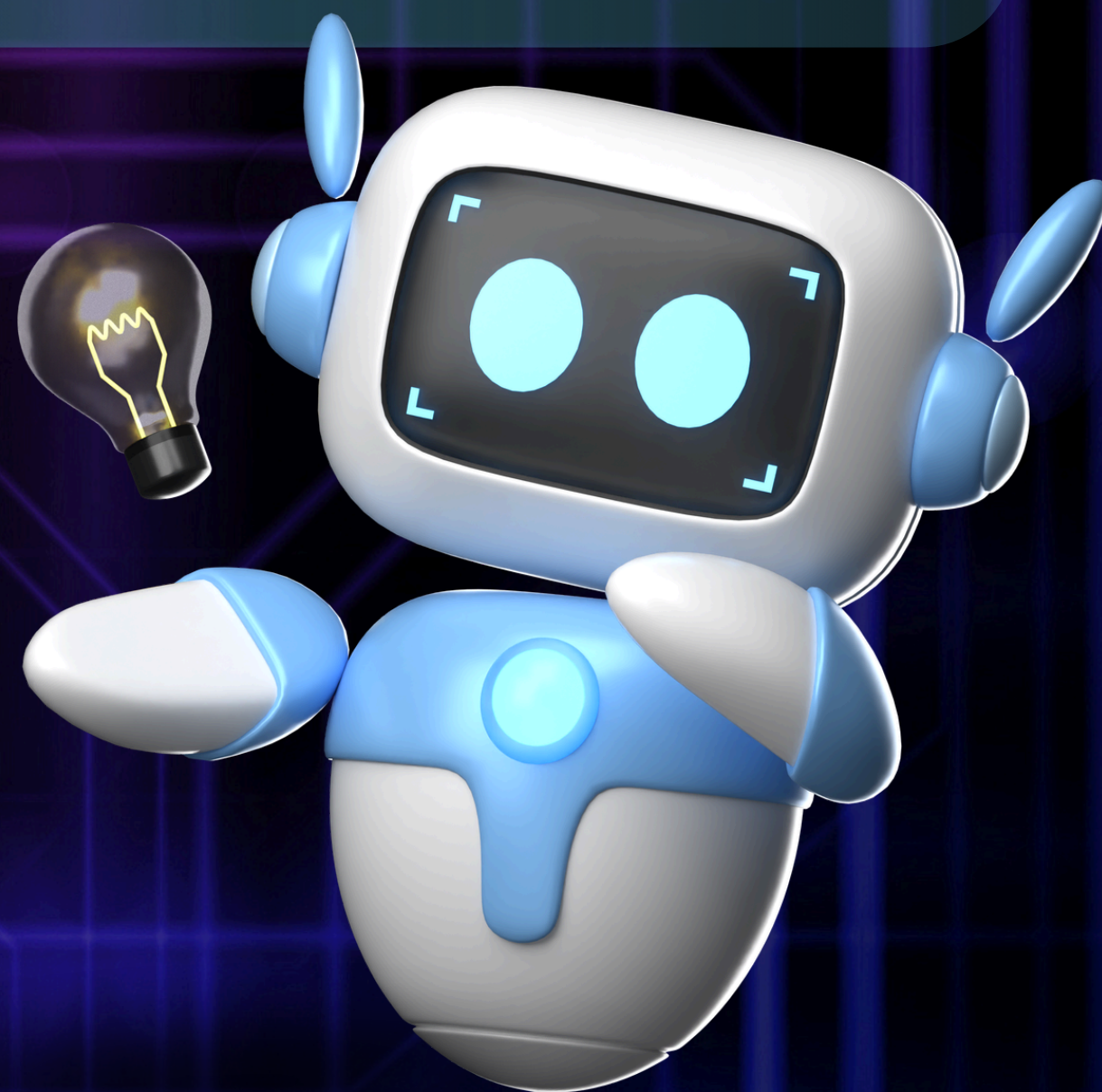


หน่วยที่ 5

การสร้าง Windows Forms Application



1. อธิบายความหมายและองค์ประกอบของ Windows Forms Application
2. สร้างโปรเจกต์ Windows Forms Application ด้วย Visual Studio
3. เลือกใช้คอนโทรลพื้นฐาน (Label, TextBox, Button ฯลฯ)
4. ปรับแต่งคุณสมบัติผ่าน Properties Window
5. เขียนโค้ดจัดการเหตุการณ์ (Event Handling)
6. ประยุกต์ใช้ตัวแปร/เงื่อนไข/ฟังก์ชันร่วมกับ Windows Forms



หน่วยที่ 5 การสร้าง Windows Forms Application

วัตถุประสงค์เชิงพฤติกรรม 6 ข้อ

- 1. อธิบายความหมายและองค์ประกอบของ Windows Forms Application
- 2. สร้างโปรเจกต์ Windows Forms Application ด้วย Visual Studio
- 3. เลือกใช้คอนโทรลพื้นฐาน (Label, TextBox, Button ฯลฯ)
- 4. ปรับแต่งคุณสมบัติผ่าน Properties Window
- 5. เขียนโค้ดจัดการเหตุการณ์ (Event Handling)
- 6. ประยุกต์ใช้ตัวแปร/เงื่อนไข/ฟังก์ชันร่วมกับ Windows Forms

5.1 ความหมายและองค์ประกอบของ Windows Forms Application

5.1.1 ความหมายของ Windows Forms Application

Windows Forms Application คือโปรแกรมประเภทหนึ่งในกลุ่มเทคโนโลยี .NET ที่ใช้สำหรับพัฒนาโปรแกรมที่มีส่วนติดต่อผู้ใช้แบบกราฟิก (Graphical User Interface: GUI) บนระบบปฏิบัติการ Windows โดยแสดงผลเป็นหน้าต่าง (Window) ที่ประกอบด้วยองค์ประกอบต่าง ๆ เช่น ปุ่มกด กล่องข้อความ ป้ายข้อความ ที่ผู้ใช้สามารถคลิก พิมพ์ หรือโต้ตอบได้โดยตรงด้วยเมาส์และคีย์บอร์ด

คำว่า "Forms" ในที่นี้หมายถึง แบบฟอร์ม หรือหน้าต่างโปรแกรม ซึ่งทำหน้าที่เป็นพื้นผิว (Canvas) สำหรับวางองค์ประกอบต่าง ๆ ของส่วนติดต่อผู้ใช้ โดยเบื้องหลังนั้น Windows Forms สร้างขึ้นจากไลบรารีของ .NET ชื่อ System.Windows.Forms ซึ่งมีคลาสสำเร็จรูปจำนวนมากให้เรียกใช้งาน

5.1.2 เปรียบเทียบกับ Console Application ที่เคยศึกษามา

จากหน่วยก่อนหน้า ผู้เรียนเขียนโปรแกรมประเภท Console Application ซึ่งโต้ตอบกับผู้ใช้ผ่านการพิมพ์ข้อความเข้า-ออกทางหน้าจอสีดำนั่น แต่ Windows Forms Application เปลี่ยนรูปแบบการโต้ตอบทั้งหมดให้เป็นภาพกราฟิกที่เป็นมิตรต่อผู้ใช้งานขึ้น

หัวข้อเปรียบเทียบ	Console Application	Windows Forms Application
รูปแบบการแสดงผล	ข้อความในหน้าต่างคอนโซล (Text-based)	หน้าต่างกราฟิก (GUI)
การรับข้อมูล	พิมพ์ผ่านคีย์บอร์ด (Console.ReadLine())	คลิกปุ่ม พิมพ์ในกล่องข้อความ เลือกตัวเลือกต่าง ๆ
ความเหมาะสม	เรียนรู้พื้นฐานภาษา ทดสอบอัลกอริทึม	โปรแกรมใช้งานจริงที่ต้องการหน้าต่างสวยงาม ใช้งานง่าย
ไลบรารีหลักที่ใช้	System	System.Windows.Forms

ต่อไปนี้เป็นแผนภาพเปรียบเทียบรูปแบบการโต้ตอบระหว่าง Console Application กับ Windows Forms Application

Console Application

กรอกอายุ: 25_

คุณเป็นเยาวชน

โต้ตอบด้วยการพิมพ์ข้อความเท่านั้น

Windows Forms Application

กรอกอายุของคุณ:

ตรวจสอบ

ผลลัพธ์: คุณเป็นเยาวชน

โต้ตอบด้วยการคลิก พิมพ์ในช่อง และปุ่มกด

5.1.3 องค์ประกอบหลักของ Windows Forms Application

โปรแกรม Windows Forms Application ทุกโปรแกรมประกอบด้วยองค์ประกอบสำคัญ 4 ส่วน ซึ่งทำงานประสานกันดังนี้

① Form (ฟอร์ม)

Form คือหน้าต่างหลักของโปรแกรม ทำหน้าที่เป็นพื้นที่วางองค์ประกอบต่าง ๆ ของส่วนติดต่อผู้ใช้ทั้งหมด โปรแกรมหนึ่งโปรแกรมอาจมีได้หลายฟอร์ม (เช่น หน้าล็อกอิน หน้าหลัก หน้าตั้งค่า) แต่ในระดับเริ่มต้นมักใช้เพียงฟอร์มเดียวชื่อ Form1

② Control (คอนโทรล)

Control คือองค์ประกอบสำเร็จรูปที่วางอยู่บนฟอร์ม เช่น Label, TextBox, Button ทำหน้าที่รับข้อมูล แสดงผล หรือรับคำสั่งจากผู้ใช้ ซึ่งจะได้ศึกษารายละเอียดในหัวข้อ 4.3

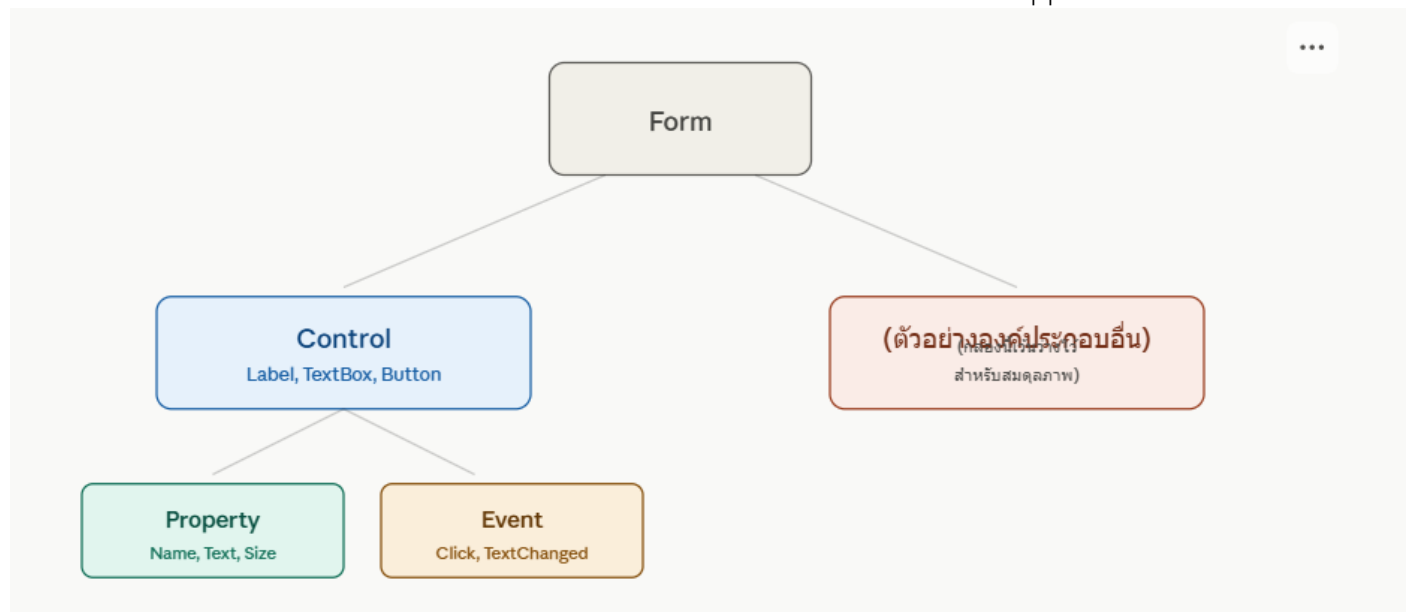
③ Property (คุณสมบัติ)

Property คือค่าที่กำหนดลักษณะของฟอร์มหรือคอนโทรล เช่น สี ขนาด ข้อความที่แสดง ตำแหน่ง ซึ่งปรับแต่งได้ผ่านหน้าต่าง Properties Window โดยไม่ต้องเขียนโค้ด

④ Event (เหตุการณ์)

Event คือการกระทำที่เกิดขึ้นขณะโปรแกรมทำงาน เช่น การคลิกปุ่ม การพิมพ์ข้อความ ซึ่งผู้เขียนโปรแกรมสามารถเขียนโค้ดตอบสนองต่อเหตุการณ์เหล่านี้ได้ (Event Handling)

ต่อไปนี้เป็นแผนภาพแสดงความสัมพันธ์ขององค์ประกอบทั้ง 4 ส่วนของ Windows Forms Application



5.1.4 ตัวอย่างประกอบ: การแกะองค์ประกอบจากโปรแกรมจริง

เพื่อให้เห็นภาพชัดเจนยิ่งขึ้น ลองพิจารณาโปรแกรมตรวจสอบช่วงวัยอย่างง่าย ซึ่งประกอบด้วยองค์ประกอบทั้ง 4 ส่วนที่กล่าวมาข้างต้น

องค์ประกอบ	ตัวอย่างในโปรแกรมนี้นี้
Form	หน้าต่างโปรแกรมชื่อ Form1 ที่บรรจุทุกอย่างไว้ภายใน
Control	Label แสดงคำว่า "กรอกอายุของคุณ:", TextBox สำหรับพิมพ์อายุ, Button "ตรวจสอบ", Label แสดงผลลัพธ์
Property	TextBox มี Property Name = txtAge, Button มี Property Text = "ตรวจสอบ"
Event	เมื่อผู้ใช้คลิก (Click) ปุ่ม "ตรวจสอบ" จะเกิด Event ที่ทริกเกอร์ให้โค้ดทำงาน

```
private void btnCheck_Click(object sender, EventArgs e) // Event: การคลิกปุ่ม
{
    int age = int.Parse(txtAge.Text); // อ่านค่าจาก Control (TextBox) ผ่าน Property (Text)

    if (age < 60)
    {
        lblResult.Text = "คุณเป็นเยาวชน"; // เปลี่ยน Property (Text) ของ Control (Label)
    }
    else
    {
        lblResult.Text = "คุณเป็นผู้สูงอายุ";
    }
}
```

คำอธิบาย: จากโค้ดนี้จะเห็นว่า **Form** (Form1) เป็นตัวบรรจุทุกอย่าง ภายในมี **Control** หลายตัว (txtAge, btnCheck, lblResult) แต่ละ Control มี **Property** ที่กำหนดลักษณะ (เช่น .Text) และเมื่อผู้ใช้คลิกปุ่มจะเกิด **Event** (Click) ที่ส่งให้โค้ดในเมธอด btnCheck_Click ทำงาน ซึ่งทั้งหมดนี้คือกลไกพื้นฐานที่สุดของการเขียนโปรแกรม Windows Forms Application

5.1.5 วงจรการทำงานของโปรแกรม Windows Forms

ต่างจาก Console Application ที่ทำงานตามลำดับคำสั่งตั้งแต่บนลงล่างแล้วจบโปรแกรม โปรแกรม Windows Forms จะทำงานแบบ **"ขับเคลื่อนด้วยเหตุการณ์"** (Event-Driven Programming) กล่าวคือ เมื่อฟอร์มแสดงผลขึ้นมาแล้ว โปรแกรมจะรออยู่เฉย ๆ จนกว่าผู้ใช้จะกระทำการบางอย่าง (เช่น คลิกปุ่ม) จึงจะทำงานตามโค้ดที่เขียนไว้ในส่วนนั้น แล้วกลับไปรออีกครั้ง วนเช่นนี้ไปเรื่อย ๆ จนกว่าผู้ใช้จะปิดโปรแกรม

ข้อสังเกตสำคัญ: แนวคิด Event-Driven นี้เป็นจุดเปลี่ยนสำคัญจากที่ผู้เรียนคุ้นเคยมาตลอด เพราะโปรแกรม Console จะทำงาน "ตามลำดับที่กำหนดไว้ล่วงหน้าทั้งหมด" แต่โปรแกรม Windows Forms จะทำงาน "ตามการกระทำของผู้ใช้ ณ ขณะนั้น" ซึ่งไม่สามารถคาดเดาลำดับล่วงหน้าได้ทั้งหมด

5.2 การสร้างโปรเจกต์ Windows Forms Application

5.2.1 ขั้นตอนการสร้างโปรเจกต์ใหม่ที่ละเอียด

การสร้างโปรเจกต์ Windows Forms Application ในโปรแกรม Visual Studio มีขั้นตอนที่ชัดเจนดังนี้

ขั้นตอนที่ 1: เปิดโปรแกรม Visual Studio และเลือกสร้างโปรเจกต์ใหม่

เมื่อเปิดโปรแกรม Visual Studio จะพบหน้าจอเริ่มต้น (Start Window) ให้คลิกที่ **Create a new project**

ขั้นตอนที่ 2: เลือกเทมเพลตโปรเจกต์

ในหน้าต่างเลือกเทมเพลต ให้กรองประเภทภาษาเป็น **C#** แล้วเลือกเทมเพลตชื่อ **Windows Forms App** จากรายการที่แสดง สังเกตคำอธิบายด้านข้างที่ระบุว่า "โปรเจกต์สำหรับสร้างแอปพลิเคชันที่มีส่วนติดต่อผู้ใช้แบบ Windows Forms"

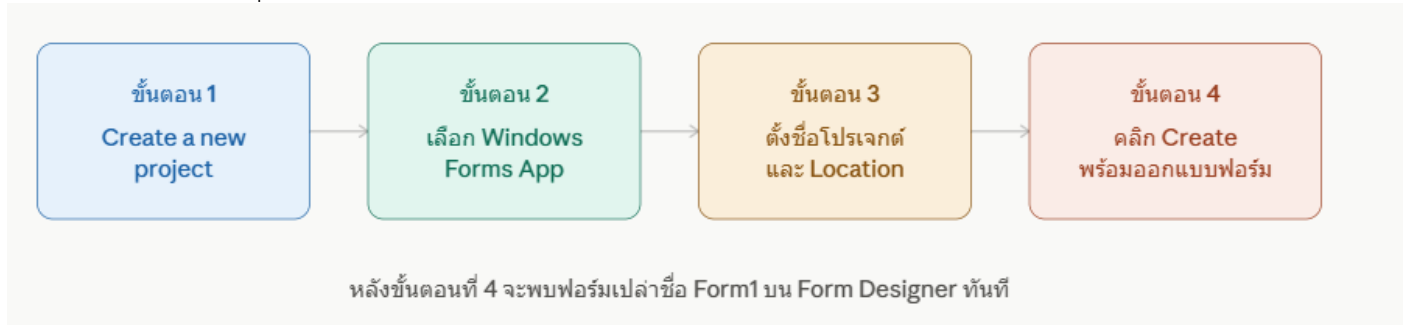
ขั้นตอนที่ 3: ตั้งชื่อโปรเจกต์และตำแหน่งจัดเก็บ

กำหนดข้อมูลของโปรเจกต์ 3 รายการ

รายการ	ความหมาย	ตัวอย่าง
Project name	ชื่อโปรเจกต์/โปรแกรมที่จะสร้าง	CheckAgeApp
Location	ตำแหน่งโฟลเดอร์ที่จะจัดเก็บไฟล์โปรเจกต์	D:\โปรแกรมC#\
Solution name	ชื่อของ Solution (ปกติจะตั้งตามชื่อโปรเจกต์อัตโนมัติ)	CheckAgeApp

ขั้นตอนที่ 4: คลิก Create เพื่อสร้างโปรเจกต์

เมื่อกดปุ่ม **Create** โปรแกรมจะสร้างไฟล์และโฟลเดอร์ที่จำเป็นทั้งหมดโดยอัตโนมัติ แล้วเปิดเข้าสู่หน้าจอหลัก (IDE) พร้อมแสดงฟอร์มเปล่าชื่อ Form1 บน Form Designer ทันที
ต่อไปนี้เป็นแผนภาพสรุปขั้นตอนการสร้างโปรเจกต์ทั้ง 4 ขั้นตอน



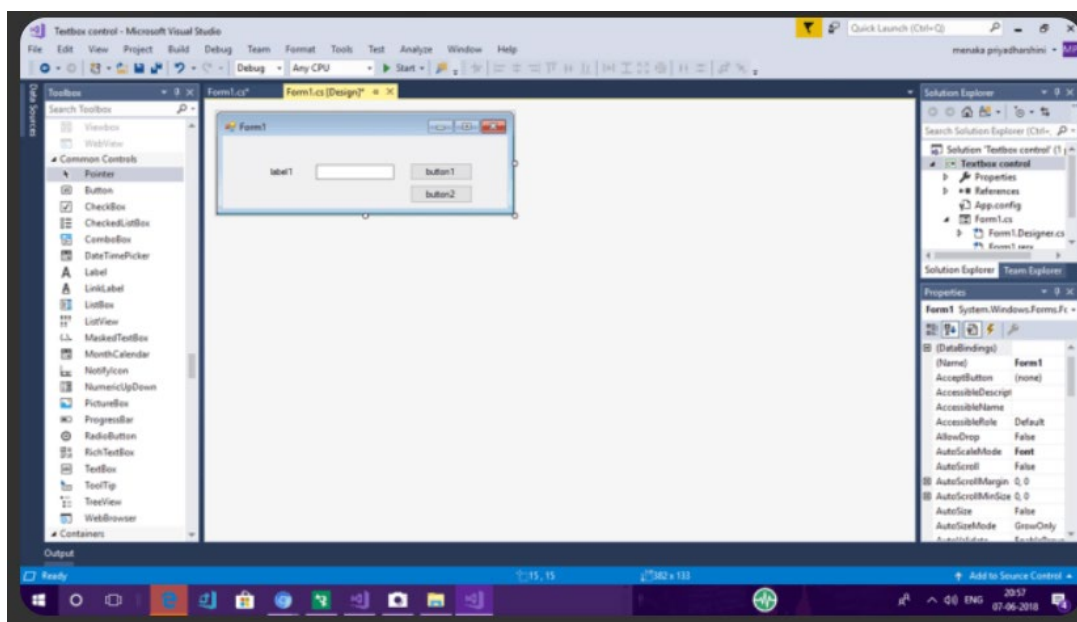
5.2.2 โครงสร้างไฟล์ที่เกิดขึ้นหลังสร้างโปรเจกต์

เมื่อสร้างโปรเจกต์เสร็จแล้ว หน้าต่าง **Solution Explorer** จะแสดงไฟล์สำคัญที่ถูกสร้างขึ้นโดยอัตโนมัติ ดังนี้

ไฟล์	หน้าที่
Form1.cs	ไฟล์โค้ดหลักของฟอร์ม เก็บคำสั่ง Event Handler ทั้งหมด
Form1.Designer.cs	ไฟล์ที่ Visual Studio สร้างอัตโนมัติ เก็บโค้ดกำหนดตำแหน่งและคุณสมบัติของคอนโทรลที่ลากวางบนฟอร์ม (ไม่ควรแก้ไขด้วยมือโดยตรง)
Program.cs	ไฟล์จุดเริ่มต้นของโปรแกรม (มีเมธอด Main() ที่สั่งเปิดฟอร์ม Form1)

ข้อสังเกตสำคัญ: ผู้เรียนสังเกตได้ว่าไฟล์ Program.cs ยังคงมีเมธอด Main() เช่นเดียวกับ Console Application ที่เคยศึกษามา เพียงแต่ภายในเมธอด Main() ของ Windows Forms Application จะมีคำสั่งสั่งให้เปิดฟอร์มขึ้นมาแสดงผลแทนที่จะเป็นคำสั่ง Console.WriteLine() เหมือนเดิม แสดงให้เห็นว่าโครงสร้างพื้นฐานของโปรแกรม C# ที่เรียนมาตั้งแต่ต้นยังคงเป็นรากฐานเดียวกัน

5.3 คอนโทรลพื้นฐานที่ใช้อยู่



ที่มา <https://www.c-sharpcorner.com/article/working-with-textbox-control-in-windows-forms-application-using-visual-studio-2/>

5.3.1 คอนโทรล 8 ชนิดที่ใช้บ่อยที่สุด

จากหัวข้อ 4.1 ผู้เรียนได้ทราบแล้วว่า Control คือองค์ประกอบที่วางอยู่บนฟอร์ม ในหัวข้อนี้จะอธิบายรายละเอียดของคอนโทรลแต่ละชนิดพร้อมตัวอย่างการใช้งานจริง

คอนโทรล	หน้าที่การใช้งาน	ตัวอย่างสถานการณ์ที่ใช้
Label	แสดงข้อความคงที่ ไม่สามารถแก้ไขได้ขณะโปรแกรมทำงาน	หัวข้อ คำอธิบาย ผลลัพธ์
TextBox	ให้ผู้ใช้งานพิมพ์ข้อมูลเข้า หรือแสดงผลลัพธ์ที่แก้ไขได้	ช่องกรอกชื่อ อายุ รหัสผ่าน
Button	สั่งให้โปรแกรมทำงานเมื่อผู้ใช้คลิก	ปุ่มคำนวณ ปุ่มบันทึก ปุ่มยกเลิก
CheckBox	ให้ผู้ใช้งานเลือกได้หลายตัวเลือกพร้อมกัน	เลือกวิชาที่สนใจหลายวิชา
RadioButton	ให้ผู้ใช้งานเลือกได้เพียงตัวเลือกเดียวในกลุ่มเดียวกัน	เลือกเพศ ชาย/หญิง
ComboBox	ให้ผู้ใช้งานเลือกค่าจากรายการแบบเลื่อนลง	เลือกจังหวัด เลือกระดับชั้น
ListBox	แสดงรายการข้อมูลหลายรายการให้เลือก	รายชื่อนักเรียนในห้อง
PictureBox	แสดงรูปภาพประกอบในฟอร์ม	โลโก้ รูปสินค้า

ต่อไปนี้เป็นภาพจำลองฟอร์มตัวอย่างที่ประกอบด้วยคอนโทรลหลายชนิดพร้อมกัน เพื่อให้เห็นภาพการจัดวางจริง

ฟอร์มลงทะเบียนตัวอย่าง

ชื่อ (Label): ← TextBox

จังหวัด (Label): ▼ เลือกจังหวัด ← ComboBox

☐ สนใจวิชาคอมพิวเตอร์ ← CheckBox

☐ ชาย ☐ หญิง ← RadioButton

สมชาย ใจดี
สมหญิง รักเรียน

 ← ListBox

← Button

รูปภาพ ← PictureBox

5.3.2 วิธีการลากวางคอนโทรลบนฟอร์ม

1. เปิดหน้าต่าง **Toolbox** (หากไม่แสดง ให้ไปที่เมนู **View > Toolbox**)
2. คลิกเลือกคอนโทรลที่ต้องการ (เช่น **Button**) จากรายการใน Toolbox
3. ลากคอนโทรลนั้นมาวางบนตำแหน่งที่ต้องการบน **Form Designer**
4. ปรับขนาดและตำแหน่งได้โดยการลากที่ขอบหรือมุมของคอนโทรล

5.3.3 ตัวอย่างการเลือกใช้คอนโทรลให้เหมาะกับโจทย์

พิจารณาโจทย์: "ออกแบบฟอร์มลงทะเบียนที่ให้ผู้ใช้งานกรอกชื่อ เลือกเพศ เลือกจังหวัด และกดปุ่มยืนยัน"

ข้อมูลที่ต้องการ	คอนโทรลที่ควรเลือกใช้	เหตุผล
กรอกชื่อ	TextBox	เป็นข้อความอิสระที่ผู้ใช้พิมพ์เองได้ ไม่จำกัดค่า
เลือกเพศ (ชาย/ หญิง)	RadioButton	เลือกได้เพียงตัวเลือกเดียว และมีตัวเลือกไม่มาก
เลือกจังหวัด	ComboBox	มีตัวเลือกจำนวนมาก การใช้เมนูเลื่อนลงประหยัดพื้นที่กว่า RadioButton
ยืนยันข้อมูล	Button	ต้องการสั่งให้โปรแกรมทำงานเมื่อคลิก

หลักการเลือกใช้คอนโทรล: ควรพิจารณาจากลักษณะของข้อมูลและจำนวนตัวเลือกเป็นหลัก หากมีตัวเลือกน้อยและต้องการให้ผู้ใช้งานเห็นตัวเลือกทั้งหมดพร้อมกัน ใช้ RadioButton หรือ CheckBox แต่หากมีตัวเลือกจำนวนมาก ควรใช้ ComboBox หรือ ListBox เพื่อประหยัดพื้นที่บนฟอร์ม

5.4 การปรับแต่งคุณสมบัติผ่าน Properties Window

5.4.1 หน้าที่ของ Properties Window

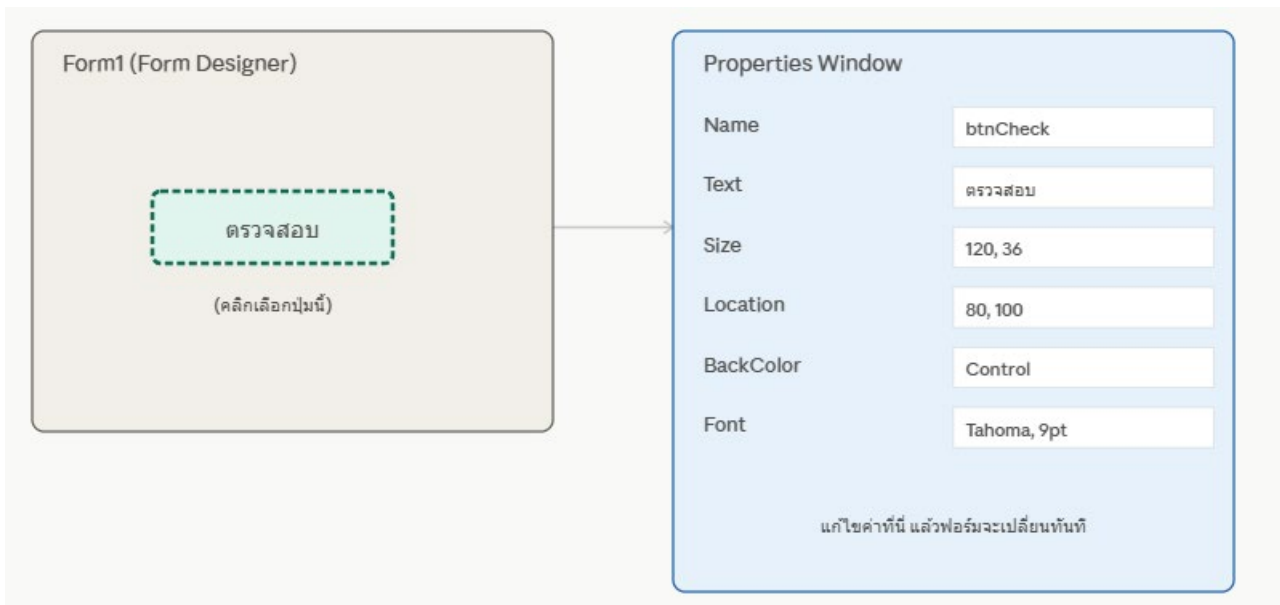
จากหัวข้อ 4.1 ผู้เรียนได้ทราบแล้วว่า **Property** คือค่าที่กำหนดลักษณะของฟอร์มหรือคอนโทรล หน้าต่าง Properties Window เป็นเครื่องมือที่ทำให้ผู้เขียนโปรแกรมสามารถปรับแต่งคุณสมบัติเหล่านี้ได้โดยไม่ต้องเขียนโค้ดสักบรรทัด เพียงคลิกเลือกคอนโทรลที่ต้องการบนฟอร์ม แล้วปรับค่าที่ต้องการในหน้าต่างนี้

ตำแหน่งของ Properties Window: โดยปกติจะอยู่มุมล่างขวาของหน้าจอ IDE หากไม่ปรากฏ สามารถเปิดได้จากเมนู View > Properties Window หรือกดปุ่ม F4

4.4.2 คุณสมบัติที่ใช้บ่อยที่สุด 6 รายการ

คุณสมบัติ	ความหมาย	ตัวอย่างค่า
Name	ชื่ออ้างอิงคอนโทรลในโค้ด (ควรตั้งชื่อให้สื่อความหมาย)	btnCheck, txtAge, lblResult
Text	ข้อความที่แสดงบนคอนโทรล	"ตรวจสอบ", "กรอกอายุ"
Size	ขนาดความกว้างและความสูงของคอนโทรล (หน่วยพิกเซล)	100, 30
Location	ตำแหน่งของคอนโทรลบนฟอร์ม (พิกัด X, Y จากมุมซ้ายบน)	50, 80
BackColor	สีพื้นหลังของคอนโทรล	สีขาว, สีฟ้าอ่อน
Font	รูปแบบและขนาดตัวอักษร	Tahoma, 12pt

ต่อไปนี้เป็นแผนภาพแสดงความสัมพันธ์ระหว่างการคลิกเลือกคอนโทรลกับการแสดงผลใน Properties Window



5.4.3 หลักการตั้งชื่อ (Name) ที่ดี

คุณสมบัติ **Name** มีความสำคัญมากที่สุด เพราะเป็นชื่อที่ใช้อ้างอิงคอนโทรลนั้นในโค้ด C# นิยมใช้ตัวย่อของชนิดคอนโทรลนำหน้าชื่อ เพื่อให้อ่านโค้ดได้ง่ายขึ้นและรู้ทันทีว่าตัวแปรนั้นอ้างอิงถึงคอนโทรลชนิดใด

ชนิดคอนโทรล	ตัวย่อที่นิยม	ตัวอย่างชื่อ
Label	lbl	lblResult, lblAge
TextBox	txt	txtName, txtAge
Button	btn	btnSubmit, btnCheck
CheckBox	chk	chkAgree
RadioButton	rdo	rdoMale, rdoFemale
ComboBox	cmb	cmbProvince

ตัวอย่างเปรียบเทียบ: หากตั้งชื่อคอนโทรลเป็น button1, textBox1 (ค่าเริ่มต้นที่ระบบตั้งให้อัตโนมัติ) โค้ดจะอ่านยากเมื่อมีคอนโทรลจำนวนมาก แต่หากเปลี่ยนเป็น btnCheck, txtAge จะทำให้อ่านโค้ดเข้าใจง่ายขึ้นทันที โดยเฉพาะเมื่อกลับมาแก้ไขโปรแกรมภายหลัง

5.4.4 ตัวอย่างการปรับแต่งคุณสมบัติทีละขั้นตอน

สมมติต้องการปรับแต่งปุ่ม Button ที่เพิ่งลากวางบนฟอร์ม ให้มีลักษณะเป็นปุ่ม "ตรวจสอบ" สีเขียว มีขั้นตอนดังนี้

1. คลิกเลือกปุ่ม Button บนฟอร์ม (จะเห็นจุดสี่เหลี่ยมเล็ก ๆ ล้อมรอบคอนโทรลที่เลือกอยู่)
2. ที่ Properties Window เปลี่ยนค่า **Name** จาก button1 เป็น btnCheck
3. เปลี่ยนค่า **Text** จาก button1 เป็น ตรวจสอบ
4. เปลี่ยนค่า **BackColor** เป็นสีเขียวอ่อน โดยคลิกที่ช่องค่าแล้วเลือกจากจานสี
5. เปลี่ยนค่า **Font** ให้มีขนาดใหญ่ขึ้นเพื่อให้อ่านง่าย

ผลลัพธ์: ปุ่มบนฟอร์มจะเปลี่ยนรูปลักษณ์ทันทีตามค่าที่ปรับ โดยไม่ต้องรันโปรแกรมหรือเขียนโค้ดใด ๆ เลย ซึ่งเป็นข้อดีสำคัญของการพัฒนาโปรแกรมแบบ Visual Design

4.5 การเขียนโค้ดจัดการเหตุการณ์ (Event Handling)

4.5.1 หลักการของ Event Handling

จากหัวข้อ 4.1 ผู้เรียนได้ทราบแล้วว่า Windows Forms ทำงานแบบ **Event-Driven** คือรอให้ผู้ใช้กระทำการบางอย่างก่อน จึงจะทำงานตามโค้ดที่เขียนไว้ **Event Handler** คือเมธอดพิเศษที่ถูกเขียนขึ้นเพื่อตอบสนองต่อเหตุการณ์เฉพาะอย่างของคอนโทรลใดคอนโทรลหนึ่ง

5.5.2 วิธีการสร้าง Event Handler

1. ดับเบิลคลิกที่คอนโทรล (เช่น Button) บน Form Designer
2. โปรแกรมจะสร้างเมธอดจัดการเหตุการณ์ให้อัตโนมัติ พร้อมเปิดหน้าต่าง Code Editor โดยเคอร์เซอร์อยู่ในเมธอดนั้นทันที
3. เขียนโค้ดที่ต้องการให้ทำงานเมื่อเกิดเหตุการณ์นั้นลงในเมธอด

```
private void btnCheck_Click(object sender, EventArgs e)
{
    // คำสั่งที่ต้องการให้ทำงานเมื่อคลิกปุ่มนี้
}
```

คำอธิบายชื่อเมธอด: ชื่อเมธอดจะถูกตั้งอัตโนมัติตามรูปแบบ ชื่อคอนโทรล_ชื่อเหตุการณ์ เช่น btnCheck_Click หมายถึงเมธอดที่ทำงานเมื่อมีการคลิก (Click) ปุ่มชื่อ btnCheck

5.5.3 เหตุการณ์ (Event) พื้นฐานที่พบบ่อย

เหตุการณ์	เกิดขึ้นเมื่อใด	คอนโทรลที่ใช้บ่อย
Click	ผู้ใช้คลิกที่คอนโทรล	Button
TextChanged	ข้อความในคอนโทรลมีการเปลี่ยนแปลง	TextBox
CheckedChanged	สถานะการเลือกเปลี่ยนแปลง	CheckBox, RadioButton
SelectedIndexChanged	ผู้ใช้เลือกรายการใหม่	ComboBox, ListBox
Load	ฟอร์มถูกเปิดขึ้นมาครั้งแรก	Form

ต่อไปนี้เป็นแผนภาพแสดงลำดับการทำงานเมื่อผู้ใช้คลิกปุ่ม ตั้งแต่เกิด Event จนถึงการอัปเดตหน้าจอ



ทั้ง 4 ขั้นตอนนี้เกิดขึ้นทุกครั้งที่ใช้คลิกปุ่ม แล้วโปรแกรมกลับไปรอเหตุการณ์ถัดไป

5.5.4 ตัวอย่างโปรแกรมสมบูรณ์: โปรแกรมทักทายผู้ใช้

```
using System;
using System.Windows.Forms;
namespace GreetingApp
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
    }
}
```

```

private void btnGreet_Click(object sender, EventArgs e)
{
    string name = txtName.Text;           // อ่านค่าจาก TextBox

    if (name == "")
    {
        lblResult.Text = "กรุณารอกชื่อก่อนกดปุ่ม";
    }
    else
    {
        lblResult.Text = "สวัสดี คุณ " + name + " ยินดีต้อนรับ!";
    }
}
}

```

คำอธิบาย: โค้ดนี้แสดงให้เห็นการผสมผสาน **Event Handling** (btnGreet_Click) เข้ากับความรู้เดิมเรื่องคำสั่งเงื่อนไข (if-else) ที่ผู้เรียนศึกษามาก่อนหน้า โดยตรวจสอบก่อนว่าผู้ใช้กรอกชื่อมาหรือไม่ หากไม่ได้กรอก (name == "") จะแจ้งเตือน แต่หากกรอกมาแล้วจะแสดงข้อความทักทาย ซึ่งเป็นรูปแบบการตรวจสอบข้อมูลนำเข้า (Input Validation) ที่พบได้บ่อยมากในโปรแกรม Windows Forms จริง

5.5.5 ข้อผิดพลาดที่พบบ่อยเกี่ยวกับ Event Handling

ข้อผิดพลาด	สาเหตุ	วิธีแก้ไข
คลิกปุ่มแล้วไม่มีอะไรเกิดขึ้น	ดับเบิลคลิกผิดพลาดคอนโทรล หรือยังไม่ได้เขียนโค้ด	ตรวจสอบว่าเขียนโค้ดอยู่ในเมธอดของปุ่มที่ถูกต้อง
Error "does not exist in current context"	สะกดชื่อคอนโทรลในโค้ดไม่ตรงกับ Name ที่ตั้งไว้	ตรวจสอบ Properties Window ว่า Name ตรงกับที่เขียนในโค้ด
สร้าง Event Handler ข้ามขั้นตอนโดยไม่ตั้งใจ	ดับเบิลคลิกคอนโทรลเดิมซ้ำหลายครั้ง	ลบเมธอดที่ซ้ำออก เหลือไว้เพียงเมธอดเดียว

5.6 ตัวอย่างโปรแกรมประยุกต์

หัวข้อนี้จะรวบรวมความรู้จากหัวข้อ 4.1–4.5 ทั้งหมด (Form, Control, Property, Event) มาประยุกต์สร้างเป็นโปรแกรมที่ใช้งานได้จริง 3 ตัวอย่าง เรียงจากง่ายไปยาก

ตัวอย่างที่ 1: โปรแกรมตรวจสอบช่วงวัย (ทบทวนแบบละเอียด)

ขั้นตอนการออกแบบฟอร์ม

ลำดับ	คอนโทรลที่วาง	Name ที่ตั้ง	Text ที่กำหนด
1	Label	lblAge	"กรอกอายุของคุณ:"
2	TextBox	txtAge	(เว้นว่างไว้ให้ผู้พิมพ์)
3	Button	btnCheck	"ตรวจสอบ"
4	Label	lblResult	(เว้นว่างไว้แสดงผลลัพธ์)

โค้ดจัดการเหตุการณ์เมื่อคลิกปุ่ม "ตรวจสอบ"

```
private void btnCheck_Click(object sender, EventArgs e)
{
    int age = int.Parse(txtAge.Text);

    if (age < 60)
    {
        lblResult.Text = "คุณเป็นเยาวชน";
    }
    else
    {
        lblResult.Text = "คุณเป็นผู้สูงอายุ";
    }
}
```

ตัวอย่างที่ 2: โปรแกรมเครื่องคิดเลขอย่างง่าย

โปรแกรมนี้แสดงการใช้คอนโทรลหลายตัวร่วมกัน และการประยุกต์คำสั่ง switch-case ที่ได้ศึกษามาในหน่วยก่อนหน้าเข้ากับ Windows Forms

ขั้นตอนการออกแบบฟอร์ม

ลำดับ	คอนโทรลที่วาง	Name ที่ตั้ง	Text/หน้าที่
1	TextBox	txtNum1	ช่องกรอกตัวเลขที่ 1
2	ComboBox	cmbOperator	เลือกเครื่องหมาย (+, -, *, /)
3	TextBox	txtNum2	ช่องกรอกตัวเลขที่ 2
4	Button	btnCalculate	"คำนวณ"
5	Label	lblAnswer	แสดงผลลัพธ์

โค้ดจัดการเหตุการณ์เมื่อคลิกปุ่ม "คำนวณ"

```
private void btnCalculate_Click(object sender, EventArgs e)
{
    double num1 = double.Parse(txtNum1.Text);
    double num2 = double.Parse(txtNum2.Text);
    string op = cmbOperator.Text;
    double result = 0;
    switch (op)
    {
        case "+":
            result = num1 + num2;
            break;
        case "-":
            result = num1 - num2;
            break;
        case "*":
            result = num1 * num2;
```

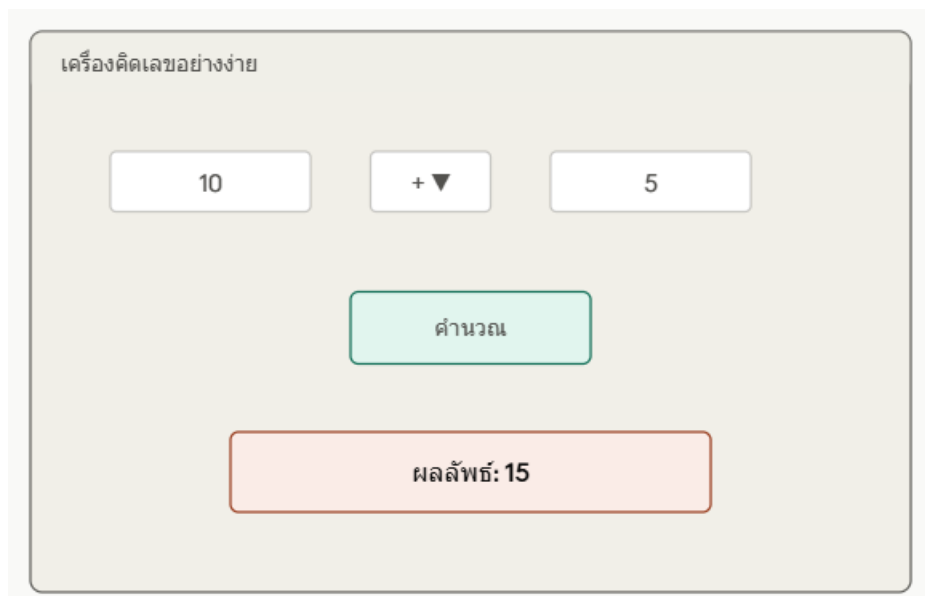
```

        break;
    case "/":
        result = num1 / num2;
        break;
    default:
        lblAnswer.Text = "กรุณาเลือกเครื่องหมาย";
        return;
    }
    lblAnswer.Text = "ผลลัพธ์: " + result;
}

```

คำอธิบาย: โปรแกรมนี้อ่านค่าตัวเลขจาก txtNum1 และ txtNum2 อ่านเครื่องหมายที่เลือกจาก cmbOperator.Text แล้วใช้คำสั่ง switch-case เลือกวิธีคำนวณตามเครื่องหมายที่เลือก คำสั่ง return; ใน default ทำหน้าที่หยุดการทำงานของเมธอดทันทีหากผู้ใช้ยังไม่ได้เลือกเครื่องหมาย

ต่อไปนี้เป็นภาพจำลองหน้าต่างของโปรแกรมเครื่องคิดเลขอย่างง่ายที่ออกแบบไว้ข้างต้น



ตัวอย่างที่ 3: โปรแกรมคำนวณดัชนีมวลกาย (BMI)

โปรแกรมนี้ประยุกต์ความรู้จากหลายหน่วยเข้าด้วยกัน ทั้งการแปลงชนิดข้อมูล การคำนวณด้วยฟังก์ชันทางคณิตศาสตร์ และการตรวจสอบเงื่อนไขหลายระดับด้วย if-else if

ขั้นตอนการออกแบบฟอร์ม

ลำดับ	คอนโทรลที่วาง	Name ที่ตั้ง	หน้าที่
1	TextBox	txtWeight	กรอกน้ำหนัก (กิโลกรัม)
2	TextBox	txtHeight	กรอกส่วนสูง (เมตร)
3	Button	btnCalcBMI	คำนวณค่า BMI
4	Label	lblBMIResult	แสดงค่า BMI และระดับการประเมิน

โค้ดจัดการเหตุการณ์เมื่อคลิกปุ่ม "คำนวณ BMI"

```
private void btnCalcBMI_Click(object sender, EventArgs e)
```

```

{
    double weight = double.Parse(txtWeight.Text);
    double height = double.Parse(txtHeight.Text);
    double bmi = weight / (height * height);
    string category;
    if (bmi < 18.5)
    {
        category = "น้ำหนักต่ำกว่าเกณฑ์";
    }
    else if (bmi < 23.0)
    {
        category = "น้ำหนักปกติ";
    }
    else if (bmi < 25.0)
    {
        category = "น้ำหนักเกิน";
    }
    else
    {
        category = "โรคอ้วน";
    }
    lblBMIResult.Text = "ค่า BMI ของคุณคือ " + Math.Round(bmi, 2) + " (" + category + ")";
}

```

คำอธิบาย: โปรแกรมนี้แสดงให้เห็นว่าความรู้เรื่องฟังก์ชันทางคณิตศาสตร์ (Math.Round()) และคำสั่ง if-else if แบบหลายเงื่อนไข ที่เคยศึกษาในหน่วยก่อนหน้า สามารถนำมาผสมผสานกับ Windows Forms ได้อย่างสมบูรณ์ กลายเป็นโปรแกรมที่ใช้งานได้จริงในชีวิตประจำวัน

ตารางสรุปการผสมผสานความรู้ในตัวอย่างทั้ง 3 โปรแกรม

ตัวอย่าง	ความรู้จากหน่วยก่อนหน้าที่นำมาใช้
ตรวจสอบช่วงวัย	การแปลงชนิดข้อมูล (int.Parse), คำสั่ง if-else
เครื่องคิดเลข	ตัวดำเนินการทางคณิตศาสตร์, คำสั่ง switch-case
คำนวณ BMI	ฟังก์ชันทางคณิตศาสตร์ (Math.Round), คำสั่ง if-else if หลายเงื่อนไข

ข้อสรุปสำคัญ: ตัวอย่างทั้ง 3 โปรแกรมนี้ยืนยันหลักการสำคัญของหน่วยที่ 4 คือ Windows Forms Application ไม่ใช่ความรู้ชุดใหม่ที่แยกขาดจากสิ่งที่เรียนมา แต่เป็นการนำตัวแปร ตัวดำเนินการ คำสั่งเงื่อนไข และฟังก์ชันที่ผู้เรียนสั่งสมมาตลอดทั้งรายวิชา มาใส่ "หน้าตา" แบบกราฟิกที่ใช้งานง่ายขึ้น แทนที่จะรับ-แสดงผลผ่าน Console เพียงอย่างเดียว